Práctica recuperar rama principal Github

Bruce Derek Peña Contreras
Compañero: Diego Gutierrez
Ficha: 3406012
Instructora: Isaura Suarez

Resumen

En la presente práctica se implementó un flujo de trabajo colaborativo utilizando Git y GitHub entre dos desarrolladores. El proceso abarcó la clonación de un repositorio remoto, la creación de ramas independientes para añadir nuevas funciones a un programa en Python (calculadora) y la fusión de cambios hacia la rama principal (`master`). Durante el proceso de integración, se dañó intencionalmente el historial de la rama `master` remota al sobrescribir el trabajo de un compañero mediante un empuje forzado. Posteriormente, el daño se revirtió utilizando comandos de restauración de historial, culminando con la correcta sincronización, resolución de conflictos locales y el envío seguro del código final.

Introducción y Objetivos

Marco Teórico

Git es un sistema de control de versiones distribuido que permite rastrear cambios en el código fuente. Cuando múltiples desarrolladores trabajan en el mismo repositorio, Git protege el historial bloqueando subidas de código si el entorno local está desactualizado. Sin embargo, comandos como `git push --force` permiten evadir estas protecciones, sobrescribiendo la línea de tiempo del servidor y borrando el trabajo de otros. Para reparar este tipo de incidentes, Git ofrece herramientas de historial como `git reset --hard`, que regresan el proyecto a un commit previo exacto. Si ocurren discrepancias al descargar cambios con `git pull`, el sistema entra en un estado de

pausa obligatoria (`|MERGING`) hasta que un humano resuelva las líneas de código contradictorias.

Objetivos

- **Objetivo General:** Aplicar un flujo de trabajo colaborativo en Git, simulando, identificando y resolviendo daños críticos en el historial de un repositorio, así como conflictos de fusión de código.
- **Objetivos Específicos:**
    1. Implementar trabajo en ramas secundarias para crear funciones matemáticas de forma aislada.
    2. Dañar intencionalmente el historial remoto de la rama `master` provocando la pérdida de commits de un compañero.
    3. Restaurar el repositorio local y remoto a una versión estable utilizando el registro de commits.
    4. Resolver un estado de bloqueo por conflicto de fusión al intentar sincronizar el código local con el del servidor.

Herramientas y Entorno

- **Sistema de Control de Versiones:** Git (CLI / Terminal tipo Bash)
- **Plataforma de Alojamiento Remoto:** GitHub
- **Editor de Código:** Visual Studio Code
- **Lenguaje del Proyecto:** Python (`calculadora.py`)
- **Equipos de cómputo:** 2 estaciones de trabajo locales (Entorno de práctica: carpeta `Taller-emergencia`).

Procedimiento

1. Creación e Inicialización del Repositorio:

- Se creó un repositorio remoto en la plataforma GitHub.
- El primer integrante realizó la creación de los archivos base de la rama master.
- El segundo integrante clonó el repositorio para posteriormente trabajar.

2. Desarrollo en Ramas Independientes:

- Se creó y desplegó una rama de trabajo propia para aislar los cambios del archivo `calculadora.py`:`git checkout -b bruce/ git checkout -b Diego`
- Se programaron nuevas funciones (resta, división y un menú) en la rama secundaria, mientras el compañero de equipo modificaba la rama principal de forma remota.

3. Fusión Local y Generación del Daño (Sobrescritura de Historial):

- Se guardaron y confirmaron los cambios en el repositorio local (rama `master`):`git add .git commit -m "Hice la funcion de resta y división y el menu"`
- Se fusionó la rama secundaria `bruce` hacia `master` de manera local.
- Para generar el daño, se subió el código ignorando intencionalmente la descarga previa (`git pull`) y forzando al servidor a aceptar la versión local, borrando los commits del compañero:`git push origin master --force`

4. Restauración del Historial (Reversión del Daño):

- Dado que el empuje forzado eliminó el commit del compañero en GitHub y dicho código nunca fue descargado al entorno local del primer integrante, no fue posible restaurarlo localmente.
- El compañero de equipo, quien conservaba su historial local sin alteraciones, intervino para solucionar el daño.
- El compañero volvió a subir su código al servidor, restaurando exitosamente la "Version de la calculadora estable" (commit 3cfc39b) para que volviera a ser visible y estuviera disponible en el repositorio remoto.

5. Sincronización Correcta y Resolución del Conflicto

- Una vez restaurado el servidor por el compañero, el primer integrante ejecutó la descarga del código para recuperar el trabajo perdido de forma segura:`git pull origin master`

- Al detectarse ediciones simultáneas en las mismas líneas, la terminal de Git entró en estado de bloqueo indicando `(master|MERGING)`.
- Se abrió Visual Studio Code para limpiar manualmente las etiquetas de conflicto inyectadas por Git (`<<<<<<< HEAD`, `=======`, `>>>>>>>`) en `calculadora.py`.
- Se marcó el conflicto como resuelto y se cerró el estado de fusión creando un nuevo commit final

Resultados

- **Estado Inicial:** Repositorio remoto sincronizado con dos ramas de desarrollo paralelas partiendo del mismo commit base.
- **Conflicto Detectado:** Impase durante la integración de la segunda rama a la rama `main` debido a cambios en las mismas líneas del archivo.
- **Estado Final:** Rama `main` actualizada con la versión seleccionada. El historial de commits del repositorio refleja el proceso de bifurcación, la primera integración y el commit de resolución de conflicto.

Discusión de Resultados

Estado de Daño: Se logró reemplazar con éxito la línea de tiempo del repositorio de GitHub, evidenciando que el uso de `--force` elimina de la vista pública los commits no sincronizados de otros colaboradores.

Restauración: La restauración fue exitosa únicamente porque el segundo desarrollador mantenía una copia local desvinculada del daño remoto, lo que permitió devolver el commit `3cfc39b` al servidor y reescribir la historia a su estado funcional.

Resolución final: Tras la intervención del compañero, el flujo de trabajo colaborativo se normalizó mediante un `git pull`. Se eliminó correctamente el estado de bloqueo `|MERGING` al limpiar el código fuente en el editor de texto.

Conclusiones

1. Se comprendió que el comando `git push --force` es altamente destructivo en entornos colaborativos, ya que elimina el trabajo de otros integrantes del servidor principal.
2. Se comprobó que en Git, si el servidor sufre pérdida de datos o alteraciones graves en el historial, la mejor estrategia de recuperación es utilizar el repositorio local de un compañero que no haya sido afectado por el cambio.
3. Se determinó empíricamente que Git detiene el flujo operativo (`|MERGING`) y bloquea nuevos comandos cuando detecta colisiones de código directas al hacer un `pull`.
4. Se concluye que para evitar la pérdida de información en Git es ejecutar siempre `git pull origin master` para integrar el trabajo remoto antes de intentar realizar cualquier confirmación hacia el servidor.

Repositorio

https://github.com/Bruce-coder17/Clase-22-10-26